

Программирование на С

Лекция 3

Переменные

Что мы уже умеем?

```
#include <stdio.h>

int main(void)
{
    printf("Hello, world!\n");
    return 0;
}
```

Программа умеет выводить текст на экран.

Проблема

```
#include <stdio.h>

int main(void)
{
    printf("Иван\n");
    printf("Иван\n");
    printf("Иван\n");

    return 0;
}
```

Что делать, если имя нужно изменить?

Переменные

Переменная — это именованное место, в котором хранится значение.

Переменные позволяют хранить данные и использовать их в разных местах программы.

Первая переменная

```
#include <stdio.h>

int main(void)
{
    int age = 18;

    printf("%d\n", age);

    return 0;
}
```

Из чего состоит объявление?

```
int age = 18;
```

Элемент	Значение
int	ТИП
age	ИМЯ
=	присвоить значение
18	значение
;	конец инструкции

Значение можно изменить

```
#include <stdio.h>

int main(void)
{
    int age = 18;

    printf("%d\n", age);

    age = 19;

    printf("%d\n", age);

    return 0;
}
```

Вывод

```
18
19
```

Что есть у переменной?

- имя;
- тип;
- значение.

Остальные характеристики мы изучим позже.

Хорошие имена

```
int age;  
int year;  
int studentCount;  
int file_size;
```

Плохие имена

```
int a;  
int x;  
int asd;  
int qwerty123;
```

Правила именования

- можно использовать буквы, цифры и символ `_`;
- имя не может начинаться с цифры;
- язык `C` различает большие и маленькие буквы.

Не начинайте имена с символа `_`.

Имена, начинающиеся с `_`, могут использоваться компилятором и стандартной библиотекой.

Типичная ошибка

```
#include <stdio.h>

int main(void)
{
    int age = 18;

    printf("%d\n", Age);

    return 0;
}
```

Переменные `age` и `Age` — разные.

Тип данных

Тип данных определяет, какие значения может хранить переменная.

Типы бывают разными

Пока знаем один тип `int` .

Он используется для хранения целых чисел.

Почему нужен тип?

Компилятор должен знать,

- что именно хранится в переменной;
- какие операции с этим значением допустимы.

Несколько переменных

```
#include <stdio.h>

int main(void)
{
    int age = 18;
    int year = 2026;
    int studentCount = 15;

    printf("%d\n", age);
    printf("%d\n", year);
    printf("%d\n", studentCount);

    return 0;
}
```

Значение одной переменной можно сохранить в другой

```
#include <stdio.h>

int main(void)
{
    int first = 10;
    int second = first;

    printf("%d\n", second);

    return 0;
}
```

Вывод

10

Переменные независимы

```
#include <stdio.h>

int main(void)
{
    int first = 10;
    int second = first;

    first = 20;

    printf("%d\n", first);
    printf("%d\n", second);

    return 0;
}
```

Вывод

```
20
10
```

При присваивании копируется значение, а не создаётся связь между переменными.

Значением может быть результат вычисления

```
#include <stdio.h>

int main(void)
{
    int age = 18;
    int nextAge = age + 1;

    printf("%d\n", nextAge);

    return 0;
}
```

Вывод

19

Изменение значения

```
int score = 10;  
  
score = score + 5;
```

После выполнения

```
score == 15
```

Как выполняется присваивание?

```
score = score + 5;
```

1. Вычисляется правая часть

```
score + 5 == 15
```

2. Результат записывается в переменную

```
score = 15
```

Сначала вычисляется правая часть, затем изменяется значение переменной слева.

Арифметические операции

Для переменных типа `c` `int`
доступны основные арифметические операции.

Операция	Пример
+	<code>a + b</code>
-	<code>a - b</code>
*	<code>a * b</code>
/	<code>a / b</code>
%	<code>a % b</code>

Арифметические выражения

```
#include <stdio.h>

int main(void)
{
    int a = 12;
    int b = 4;

    printf("%d\n", a + b);
    printf("%d\n", a - b);
    printf("%d\n", a * b);
    printf("%d\n", a / b);

    return 0;
}
```

Вывод

```
16
8
48
3
```

Остаток от деления

```
printf("%d\n", 17 % 5);  
printf("%d\n", 20 % 5);  
printf("%d\n", 21 % 5);
```

Вывод

```
2  
0  
1
```

Оператор `%` вычисляет остаток от деления.

Откуда берутся значения?

До сих пор мы сами задавали значения переменных.

```
int a = 10;  
int b = 20;
```

Но часто значение должно вводиться пользователем.

Ввод данных

Для ввода данных используется функция `scanf` .

```
scanf("%d", &a);
```

Функция `scanf` считывает значение, введённое с клавиатуры, и сохраняет его в переменную.

Первая программа с вводом

```
#include <stdio.h>

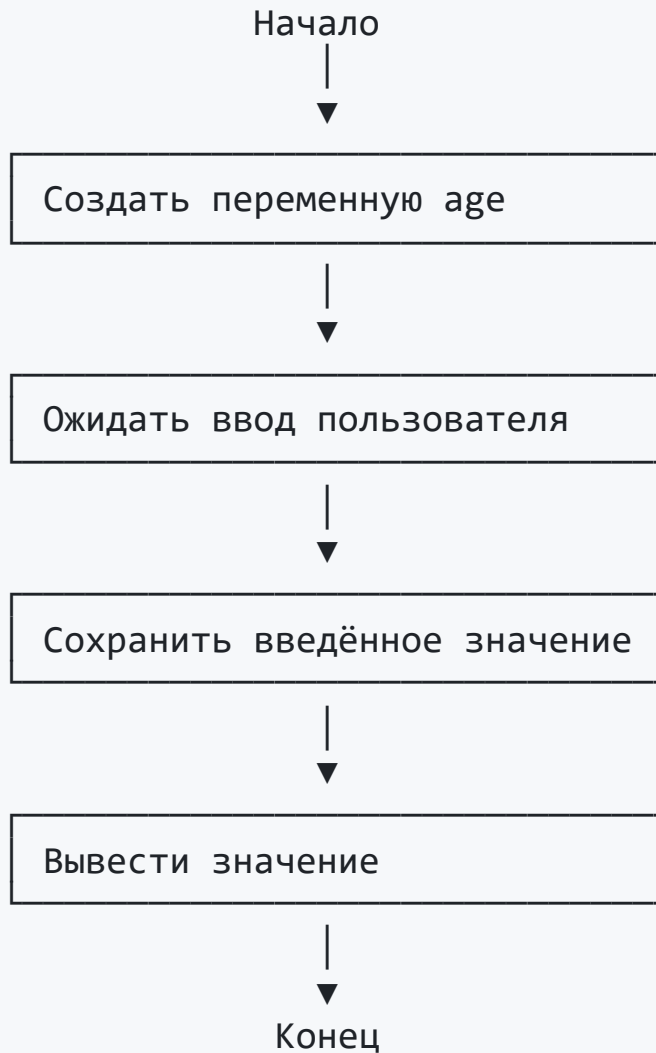
int main()
{
    int age;

    scanf("%d", &age);

    printf("%d\n", age);

    return 0;
}
```

Как работает программа?



Пример работы

```
int age;  
scanf("%d", &age);  
printf("%d\n", age);
```

Ввод

25

Вывод

25

Ввод двух чисел

```
int a;  
int b;  
  
scanf("%d", &a);  
scanf("%d", &b);  
  
printf("%d\n", a + b);
```

Ввод

7
5

Вывод

12

Используем введённые значения

```
int width;  
int height;  
  
scanf("%d", &width);  
scanf("%d", &height);  
  
int area = width * height;  
  
printf("%d\n", area);
```

Что означает %d ?

```
scanf("%d", &number);
```

%d означает, что вводится целое число.

Для чего нужен символ `&`?

```
scanf("%d", &number);
```

При работе с `scanf` перед именем переменной ставится символ `&`.

Пока достаточно запомнить это правило.

Почему он нужен, мы узнаем позже.

Типичная ошибка



```
scanf("%d", number);
```



```
scanf("%d", &number);
```

Если забыть символ `&`, программа будет работать неправильно.

Несколько значений можно вводить сразу

```
int a;  
int b;  
  
scanf("%d %d", &a, &b);  
  
printf("%d\n", a + b);
```

Ввод

7 5

Вывод

12

или

7
5